

[Home](#) / [Posts](#)

The 2026 MCP Roadmap

The updated Model Context Protocol roadmap for 2026: transport scalability, agent communication, governance maturation, and enterprise readiness, plus guidance on SEP prioritization and how to get involved.

March 9, 2026 · 6 min · David Soria Parra (Lead Maintainer)

► Table of Contents

MCP's [current spec release](#) came out in November 2025. We haven't cut a new version since, but the project hasn't stood still. Over the past year MCP has moved well past its origins as a way to wire up local tools. It now runs in production at companies large and small, powers agent workflows, and is shaped by a growing community through Working Groups, [Spec Enhancement Proposals](#) (SEPs), and a formal governance process. None of that is news, but it's the foundation we're building on.

We spent the last few months working through a long list of candidate priorities. They were informed by production experience, community feedback, and the pain points that keep surfacing. We narrowed them down to the areas that matter most for 2026. The result is an updated [roadmap document](#) that lays out where we're headed.

If you read the [January update](#), you'll recognize the broad strokes. Production deployments have different needs than the early experiments that got us here, and the roadmap now reflects that. Here's what changed and what it means for you.

From Releases to Working Groups

Previous versions of the roadmap were organized around release milestones: what's shipping in the next spec version and what comes after. That framing made sense when the project was smaller and most of the work flowed through a handful of people.

[Working and Interest Groups](#) are now the primary vehicle for protocol development, and the roadmap needed to reflect that. The new document is organized around **priority areas**, rather than around dates. Working Groups drive the timeline for their deliverables. The roadmap tells you which problems we consider most important and points you to the groups working on them.

This approach also lets us be more honest about the uncertainty inherent in a fast-growing project like MCP. A release-oriented roadmap implies a level of predictability that open-standards work rarely has.

The Priority Areas

Core maintainers ranked candidate areas, and the result was a clear top four. These are the areas where SEPs will receive expedited review and where most of our maintainer capacity is concentrated.

Transport Evolution and Scalability

Streamable HTTP is the transport that lets MCP servers run as remote services rather than local processes. It unlocked a wave of production deployments. But running it at scale has surfaced a consistent set of gaps: stateful sessions fight with load balancers, horizontal scaling requires workarounds, and there's no standard way for a registry or crawler to learn what a server does without connecting to it.

The work here falls into two parts. First, evolving the transport and session model so that servers can scale horizontally without having to hold state, as well as clear, explicit mechanisms to handle sessions. Second, a standard metadata format, that can be served via `.well-known`, so that server capabilities are discoverable without a live connection.

One thing we want to be explicit about: we are **not** adding more official transports this cycle but evolve the existing transport. Keeping the set small is a deliberate decision grounded in the [MCP design principles](#).

Agent Communication

The Tasks primitive ([SEP-1686](#)) shipped as an experimental feature and works well for what it was designed to do. Early production use has surfaced a concrete list of lifecycle gaps to close: retry semantics when a task fails transiently, and expiry policies for how long results are retained after completion.

This is the kind of iteration you can only do once something is deployed and tested in the real world. We plan to take the same approach with other parts of MCP: ship an experimental version, gather production feedback, and iterate.

Governance Maturation

Right now, every SEP requires full [Core Maintainer](#) review, regardless of domain. That's a bottleneck. It slows down Working Groups that already have the expertise to evaluate proposals in their own area.

The goal is to remove that bottleneck without sacrificing quality. Concretely, that means a documented **contributor ladder** so there's a clear path from community participant to maintainer, and a delegation model that lets trusted Working Groups accept SEPs in their domain without waiting on a full core review. Core Maintainers keep strategic oversight. Working Groups get room to move.

Enterprise Readiness

Enterprises are deploying MCP and running into a predictable set of problems: audit trails, SSO-integrated auth, gateway behavior, and configuration portability.

This is also the least defined of the four priorities, and that's intentional. We want the people experiencing these challenges to help us define the work.

A dedicated Enterprise WG does not yet exist. If you work in enterprise infrastructure and want to lead or join one, the [Working Groups page](#) explains how to get started. We also recommend joining the [contributor Discord](#) to make sure you're not duplicating work or going solo on new proposals.

We expect most of the enterprise readiness work to land as extensions rather than core spec changes. Enterprise needs are real, but they shouldn't make the base protocol heavier for everyone else.

SEP Prioritization: What It Means for Contributors

One of the most practical additions to the roadmap is explicit guidance on how SEP review capacity gets allocated.

The short version: **SEPs aligned with the priority areas above will move the fastest.** SEPs outside those areas aren't automatically rejected, but they face longer review timelines and a higher bar for justification. Maintainer bandwidth is finite, and we'd rather be transparent about where it's going.

If you're considering writing a SEP, start with the [SEP Guidelines](#). Once you're familiar with those:

1. **Check whether your proposed change maps to one of the priority areas.** If it does not, be prepared for delays in reviews.

2. **Bring it to the relevant Working Group.** SEPs that arrive with WG backing and a clear connection to the roadmap are the ones that move.

On the Horizon

Not everything we care about made the top four, and we didn't want those areas to disappear from view. We're focused on a limited set of items, but we still want protocol exploration to continue at a good pace. The roadmap now includes an **On the Horizon** section for work with real community interest, such as triggers and event-driven updates, streamed and reference-based result types, deeper security and authorization work, and maturing the extensions ecosystem.

These aren't deprioritized in the sense of "We don't want them." They're areas where we'll happily support a community-formed WG and review SEPs as time permits, but where Core Maintainers aren't actively standing things up this cycle.

Some of these already have active proposals in review, such as [SEP-1932 \(DPoP\)](#) and [SEP-1933 \(Workload Identity Federation\)](#). Others, like triggers and event-driven updates, would benefit from a new Working Group.

Get Involved

Every deliverable on the roadmap runs through a Working Group, and every Working Group is open to contributors. Here are a few ways to get involved:

- **Join a Working Group:** Working Groups are the small teams doing the actual protocol design. They meet regularly and welcome new participants. The [Working Groups & Interest Groups](#) page lists what's active and how to connect.
- **Propose a SEP:** SEPs are how changes to the protocol get proposed and reviewed. The [SEP guidelines](#) walk through the process.

- **Start an extension:** Extensions let us experiment with new capabilities outside the core spec. You can learn more in our [official Extensions documentation](#).

If you're not sure where to start, the easiest first step is to join a Working Group meeting and introduce yourself.

We're excited to build the protocol together!

[mcp](#)[roadmap](#)[governance](#)[community](#)

[Documentation](#) · [Posts](#) · [Archives](#) · [Search](#) · [Tags](#) · [GitHub](#) · [LinkedIn](#) · [RSS](#)

Copyright © Model Context Protocol a Series of LF Projects, LLC.

For web site terms of use, trademark policy and other project policies please see <https://lfprojects.org>.